

Inside _GPT - 2

Tokenization, the transformer block, and training a mini GPT. Shown on the real GPT-2, piece by piece.

"Inside GPT-2" = 5 GPT-2 tokens: 24441 · 402 · 11571 · 12 · 17

ASSIGNMENT 02 CHECK-IN

Gradio turns a Python function into a web app

```
learn = load_learner("model.pkl")
categories = learn.dls.vocab

def classify_image(image):
    prediction, index, probs = learn.predict(image)
    return dict(zip(categories, map(float, probs)))

app = gr.Interface(fn=classify_image,
                  inputs=gr.Image(),
                  outputs=gr.Label())

app.launch()
```

fn

your function: image in, probabilities out

inputs

what the user gives: an upload box for an image

outputs

what they see: the labels with probability bars

launch()

starts a small web server; in Colab the app shows up in the notebook

ASSIGNMENT 02 CHECK-IN

A Hugging Face Space is a git repo that runs your app

app.py

the Gradio code from the previous slide

requirements.txt

the same versions as Colab, CPU-only PyTorch

README.md

its header sets the SDK and the Python version

model.pkl

your exported model, plus a few example .jpg images

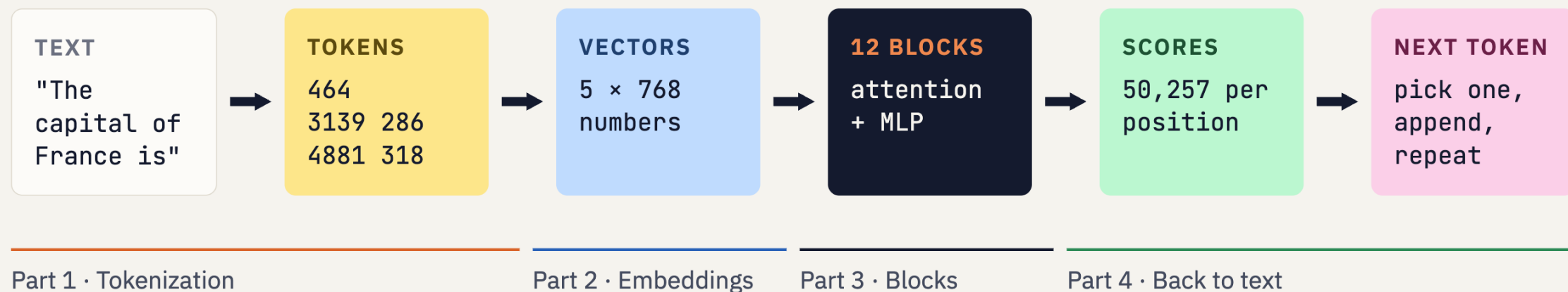
The classic trap: a model exported on Colab (Python 3.13) does not load on Python 3.10, the default of Spaces.

Fix: put `python_version: '3.13'` into the README header.

Due Oct 15, 18:00: the link to your dataset and the link to your app, via the form on Slack.

THE BIG PICTURE

An LLM is a next-token predictor



Recap of last week's video. Today we open every box of this diagram, and in Part 5 we train one from scratch.

THE BIG PICTURE

Meet GPT-2 small

50,257

tokens in the vocabulary

1,024

tokens of context it can see

768

numbers in each token vector

12

transformer blocks

12

attention heads in each block

124M

parameters (124,439,808)

OpenAI, February 2019 · trained on WebText, about 40 GB of text from 8M web pages · four sizes: 124M, 355M, 774M, 1.5B. We use the smallest: it runs on Colab even without a GPU.

THE BIG PICTURE

Why GPT-2 and not BERT?



BERT · encoder

The capital of [MASK] is Paris.

Every token sees every token. Fills in blanks;
good for classification and search.



GPT · decoder

The capital of France is ?

Each token sees only the past. Predicts the
next token, so it can write text.

ChatGPT, Claude, Llama, Gemma and Qwen are all **decoder-only** descendants of GPT-2. Since 2019 the block has changed surprisingly little.

1

Tokenization

From text to the integers the model actually sees: bytes, Byte Pair Encoding, and GPT-2's tokenizer rebuilt in 20 lines.

TOKENIZATION

How do we cut text into pieces?

Characters

36 tokens

T o k e n i z a t i o n _ i s ...

Tiny vocabulary, but very long sequences: the model spends its effort on spelling.

Words

4 tokens

Tokenization is unbelievably useful.

Short sequences, but a huge vocabulary. Typos, new names and Czech word forms are unknown.

Subwords

6 tokens (GPT-2)

Token ization _is _unbelievably _useful .

The sweet spot. Frequent words get one token, rare words are built from pieces.

TOKENIZATION

Text is just bytes

character	Unicode	UTF-8 bytes
a	97	97
ř	345	197 153
€	8364	226 130 172
🐎	128052	240 159 144 180

256

possible byte values: a vocabulary in which **no text is ever unknown**, in any language.

"Ahoj! Příliš žluťoučký kůň 🐎"

28 characters → 40 bytes

The catch: sequences get long. Tokenization glues frequent byte sequences into bigger tokens.

TOKENIZATION

Byte Pair Encoding in four steps

1. Start with the text as bytes.
2. Count all pairs of neighbours.
3. Merge the most frequent pair into a new token.
4. Repeat.

aaabdaa^aabac

11 tokens

Z^abdZ^abac

$Z = aa \cdot 9$

ZY^dZY^aac

$Y = ab \cdot 7$

X^dX^aac

$X = ZY \cdot 5$

Vocabulary = 256 bytes + one new token per merge. GPT-2: 256 + 50,000 merges + 1 special token = 50,257.

TOKENIZATION

What BPE learns first

Our BPE: 20,000 characters of Shakespeare

e_ th t_ s_ ou ,_ d_ r_ in an

Merge 19 is already a whole word: $th + e_ \rightarrow the_.$ The space sticks to the **end** of a word.

GPT-2: 50,000 merges from 40 GB of web text

_t _a he in re on _the er _s at

$_the$ is merge 7. Here the space sits at the **start** of a word, thanks to a pre-split (next slide).

GPT-2's last merges: $_Hitman$, $Commission$, $_amplification$, $_gazed$. Its longest tokens: 64 dots, 64 "=" signs, and a piece of garbled text (ÃÂÃÂ...) common enough on the web to earn a token.

TOKENIZATION

Rebuilding GPT-2's tokenizer

Step 1 · a regex splits the text into chunks

Hello 've _world 123 _how 's _are _you !!!?

Step 2 · inside a chunk, apply the earliest-learned merge

_t o k e n i z a t i o n

_to k en i z at i on after 4 merges

_to k en ization after 8

_token ization after 10: done, ids 11241 · 1634

```
our_encode(text) =  
enc.encode(text)
```

True English

True Czech

True Korean + emoji

True Python code

True arithmetic

About 20 lines of Python **are** GPT-2's tokenizer.

How GPT-2 cuts real text

E gg . I have an egg . E GG .

Capitals and the space in front change the tokens: three different "eggs".

127 + 6 77 = 8 04

Numbers are cut into arbitrary pieces. No wonder arithmetic was hard.

def f (x): ↵ ↵ ↵ if x > 0 :

In code, almost every space of the indentation is its own token. Newer tokenizers merge runs of spaces.

TOKENIZATION

The Czech token tax

English · 7

The _capital _of _France _is _Paris .

Czech · 19

H _lav n í _m ? ? st o _Franc ie _je _Pa ? ? í ? ? .

tokenizer	vocabulary	English	Czech
GPT-2	50,257	7	19
GPT-4 · cl100k_base	100,277	7	15
GPT-4o · o200k_base	200,019	7	12

GPT-2 has no token for ě, ř or ž: each is split into 2 raw bytes, shown as ?.

More tokens = a higher price, a context that fills up faster, a harder job for the model.

TOKENIZATION

Tokenization explains many LLM quirks

"How many r's in strawberry?"

▬strawberry

One single token. The model never sees the letters inside it.

Arithmetic and spelling

▬6 77 ▬8 04

Digits come in arbitrary groups.
Reversing or spelling a word is hard
for the same reason.

Glitch tokens

▬SolidGoldMagikarp

One GPT-2 token (id 43453): a
Reddit username, frequent in the
tokenizer's data but almost absent
from the model's. Early GPT-3
models could not even repeat it.

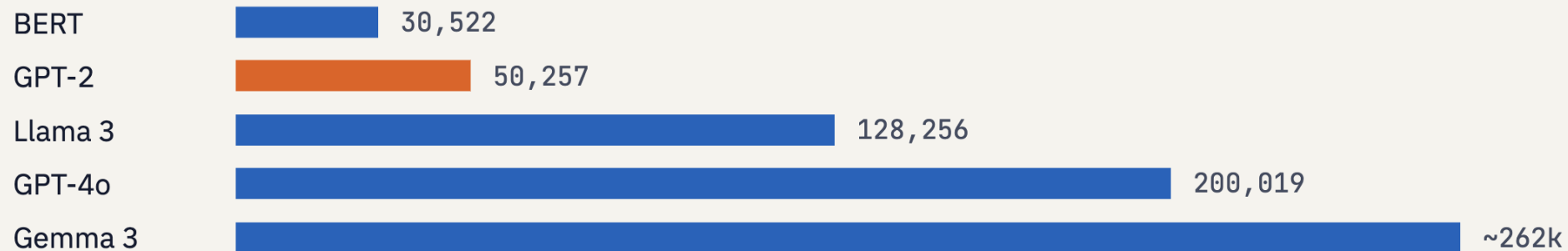
When an LLM fails at something "easy", ask first: what do the tokens look like?

TOKENIZATION

The tokenizer is a separate model



VOCABULARY SIZES KEEP GROWING



Special tokens mark structure: `<|endoftext|>` (id 50256) separates documents in GPT-2; chat models add markers like `<|im_start|>` for who is speaking.

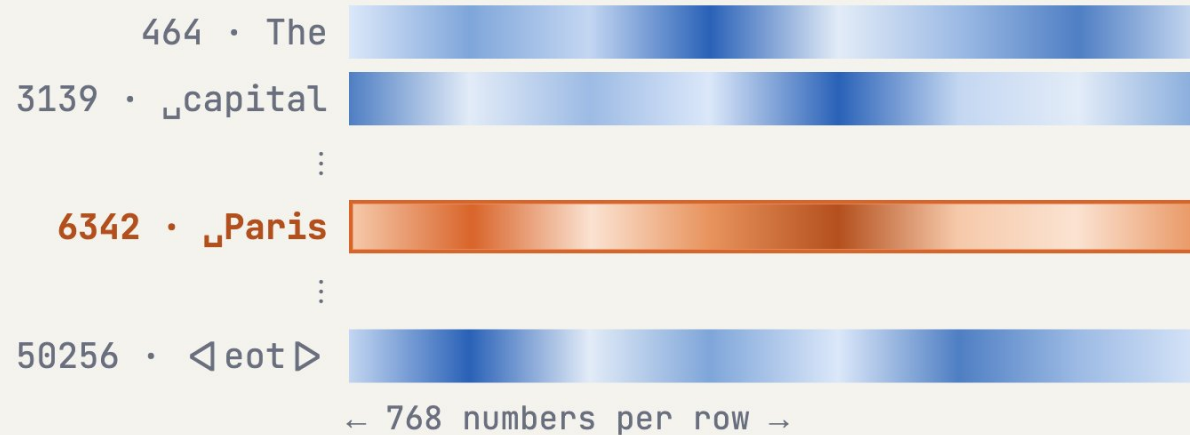
2

From tokens to vectors

A lookup table turns every token into 768 numbers, and a second table adds where the token stands.

EMBEDDINGS

The embedding is a lookup table



wte[6342]

"Look up" = take row 6342. No computation at all.

38.6M · 31 %

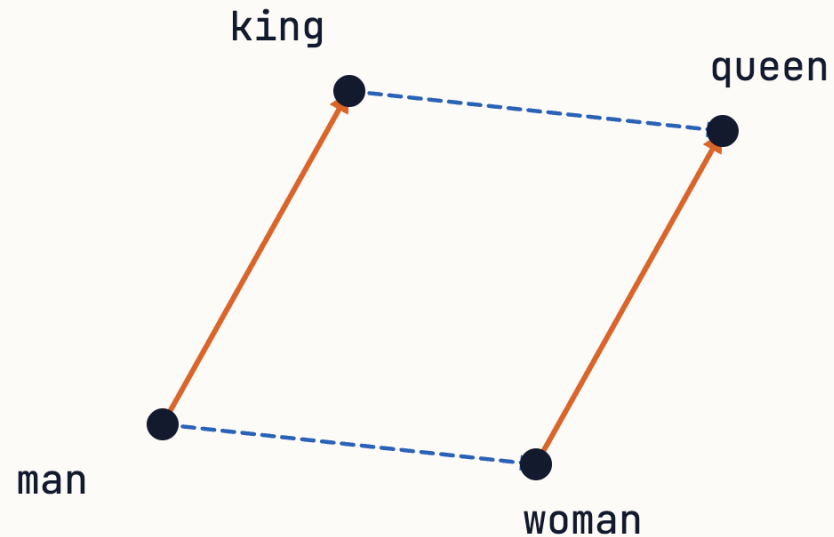
50,257 × 768 numbers: almost a third of GPT-2,
before any "thinking".

The numbers are **learned** during training, just like
the CNN kernels in Lesson 02.

EMBEDDINGS

Directions carry meaning

orange = a "royal" direction · blue = a "female" direction



$\text{king} - \text{man} + \text{woman} \approx \text{queen}$

Tokens used in similar ways get vectors that point in similar directions. Recap of the 3Blue1Brown video.

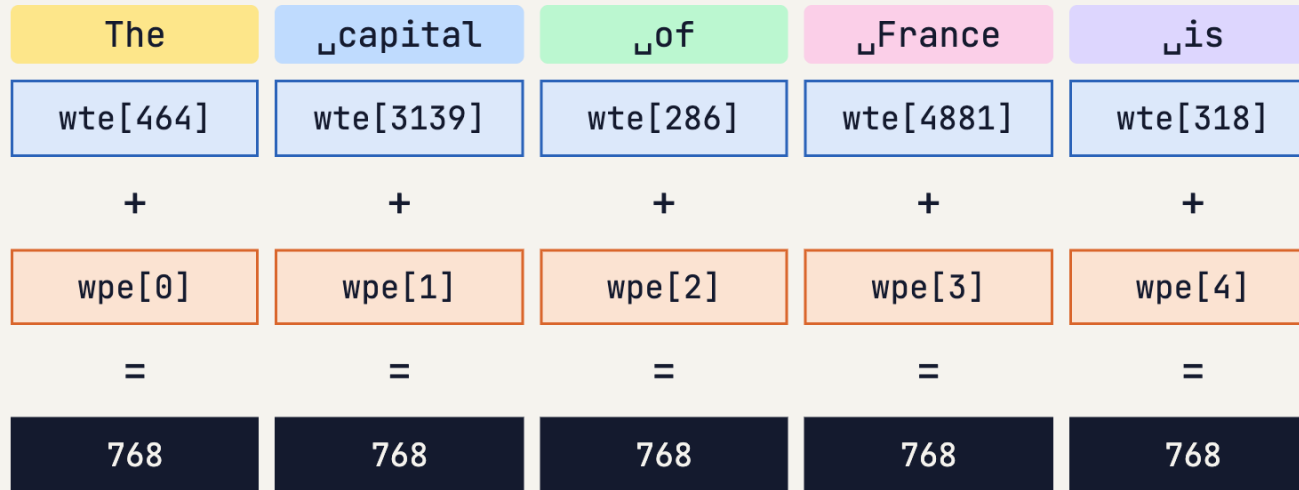
LIVE IN COLAB

```
neighbours(" dog")  
neighbours(" Paris")
```

Cosine similarity: how well do two vectors point the same way? We rank all 50,257 tokens by it.

EMBEDDINGS

Position embeddings: where does the token stand?



Attention alone does not know the order: "dog bites man" = "man bites dog".

So GPT-2 learns a second table, **wpe**: one vector for each position, $1,024 \times 768$.

There is no row 1,025: that is the context limit.

Modern models use **RoPE** instead: they rotate queries and keys by the position.

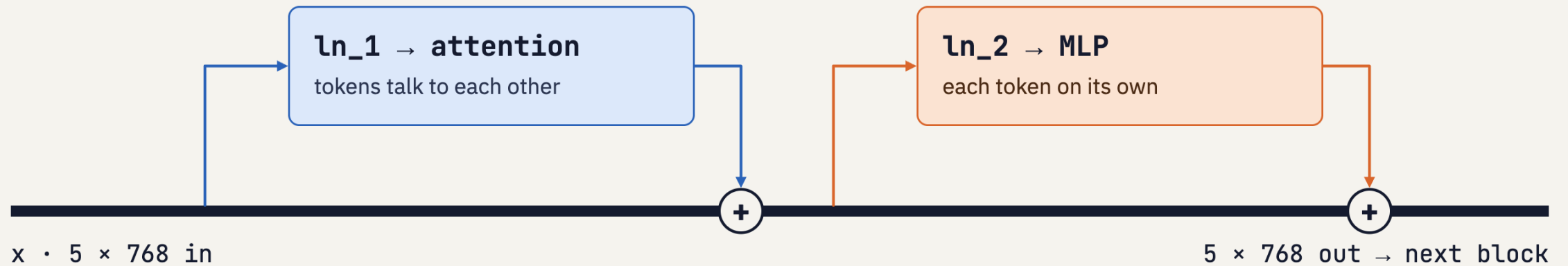
3

The transformer block

12 identical blocks. In each, the tokens first exchange information (attention), then each token works on its own (MLP).

TRANSFORMER BLOCK

Each block adds to the residual stream



```
x = x + attention(ln_1(x))  
x = x + mlp(ln_2(x))
```

A block never replaces the token vectors; it only **adds** to them. Same shape in and out, so we can stack 12 blocks.

TRANSFORMER BLOCK

Two jobs: communicate, then compute

ATTENTION

Communicate

Each token looks at the earlier tokens and copies information from them. "What is around me?"

MLP

Compute

Each token is processed on its own, by the same small network. "What do I make of it?"

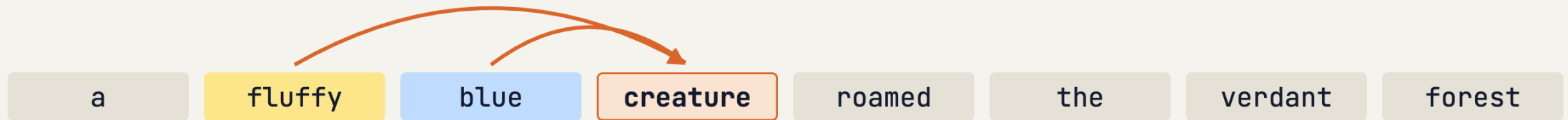
PARAMETERS OF ONE BLOCK: 7,087,872

attention 2,362,368 · $\frac{1}{3}$

MLP 4,722,432 · $\frac{2}{3}$

plus 3,072 in the two LayerNorms

Attention: the intuition



Query

"What am I looking for?" creature:
any adjectives before me?

Key

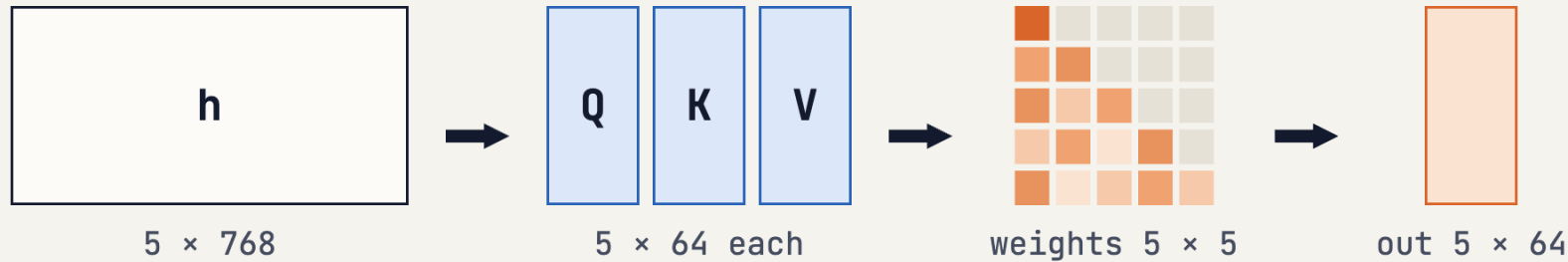
"What do I offer?" fluffy: I am
an adjective.

Value

"What I pass on if you pick me."
fluffy: soft, furry.

Score = how well my query matches your key. A high score means I take a lot of your value.

Attention: the math in one picture



```
weights = softmax( Q · KT / √64 + mask )
output  = weights · V
```

Why $\sqrt{64} = 8$? Without it the scores get large and softmax gives all the weight to a single token.

Each row of the weights sums to 1: a recipe for mixing the values of the earlier tokens.

The causal mask: no peeking

The	▯capital	▯of	▯France	▯is	
	$-\infty$	$-\infty$	$-\infty$	$-\infty$	The
		$-\infty$	$-\infty$	$-\infty$	▯capita l
			$-\infty$	$-\infty$	▯of
				$-\infty$	▯France
				$-\infty$	▯is

A token may look only at **itself and earlier tokens**. Looking ahead would be cheating: the future is what we predict.

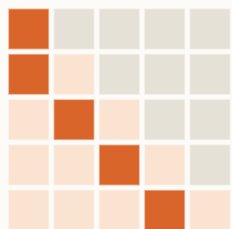
Scores above the diagonal are set to $-\infty$, and softmax turns them into 0.

Bonus: one sequence of **T** tokens gives T next-token training examples at once.

BERT, an encoder, simply has no mask.

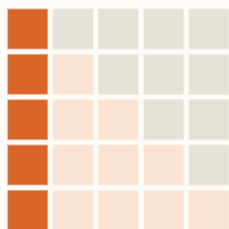
12 heads, each looking for something else

12 heads $\times$ 64 numbers = 768. Each head has its own queries, keys and values. Their outputs are glued together and mixed by one more linear layer.



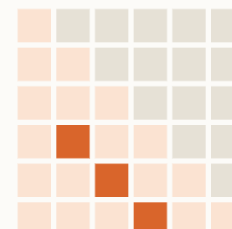
Previous token

Each token looks at the one right before it.



Attention sink

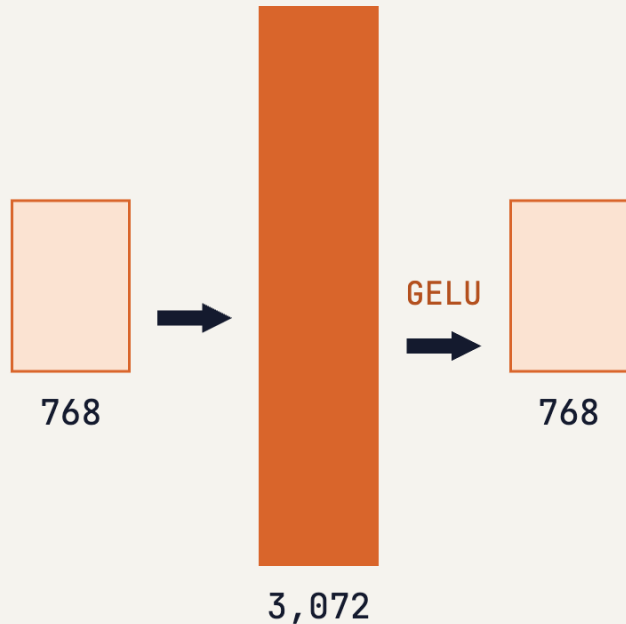
Nothing useful to look at? Park the attention on the first token.



Induction head

In "A B C A B C", look at what followed the earlier copy, to continue the pattern.

The MLP: where facts might live



Expand to 4×768 numbers, apply a non-linearity (**GELU**, a smooth cousin of ReLU), shrink back.

Applied to **each token separately**, with the same weights. No communication here.

Two thirds of the block's parameters. The 3Blue1Brown picture: "Michael Jordan" goes in, a "basketball" direction gets added.

```
mlp(h) = c_proj(gelu(c_fc(h)))
```

LayerNorm keeps the numbers sane

$$\text{ln}(x) = (x - \text{mean}) / \text{std} \cdot \text{weight} + \text{bias}$$

mean and std of the 768 numbers of one token · weight and bias are learned

Per token

Every token vector is rescaled on its own to mean 0 and spread 1.

Before each sub-block

ln_1 before attention, ln_2 before the MLP ("pre-LN"), plus a final ln_f.

Why

Without it, numbers drift through 12 blocks and training a deep stack becomes unstable.

We rebuilt GPT-2 in about 60 lines of PyTorch

```
torch.allclose(our_gpt2(ids), logits)  
True
```

Our **Block** class with GPT-2's weights copied in gives the same numbers as the real model. 7,087,872 parameters per block × 12, plus the embeddings = 124,439,808, the same count as Hugging Face.

4

Back to text

The last vectors become 50,257 scores. Pick one token, append it, and repeat.

BACK TO TEXT

Unembedding: from a vector to probabilities



Weight tying

The output uses the **same table wte** as the input. A vector that points like the embedding of `_Paris` gets a high score for `_Paris`.

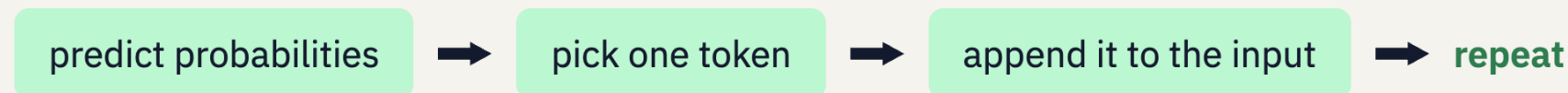
`model.lm_head.weight` is `wte` → True

LIVE IN COLAB

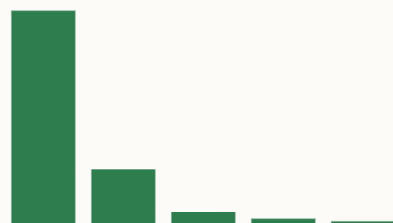
```
plot_next_token(
    "The capital of France is")
Is _Paris among the top 10?
```

BACK TO TEXT

Generating text is a loop



How we **pick** matters. The **temperature** T divides the logits before softmax:



$T = 0.5$ · sharp

$T \rightarrow 0$ always takes the top token: safe, boring, repeats itself.



$T = 1$ · the model

Sample from the model's own probabilities.



$T = 2$ · flat

More surprising choices, and more nonsense.

The logit lens: what does GPT-2 think halfway?

Apply `ln_f` and the unembedding after **every** block, not just the last one, and ask which token would win at that point.



After each block: top token and $P(_\text{Paris})$. In which block does `\_\text{Paris}` start to win?

```
for block in our_gpt2.blocks:
    x = block(x)
    peek(ln_f(x) @ wte.T)
```

5

Training a mini GPT

The same Block class, now with random weights, trained from scratch on 1 MB of Shakespeare.

TRAINING

Same blocks, random weights

	GPT-2 small	our mini GPT
blocks	12	4
vector size	768	128
heads	12	4
context	1,024 tokens	128 characters
vocabulary	50,257 tokens	65 characters
parameters	124,439,808	818,048
training data	40 GB of web	1.1 MB Shakespeare

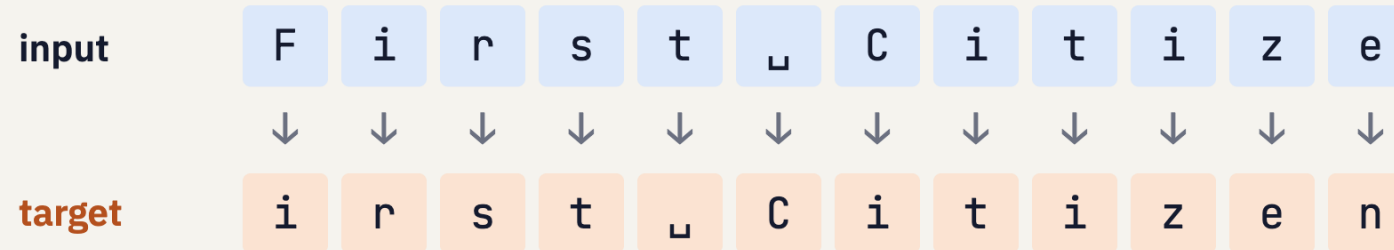
```
mini = GPT(vocab_size=65,  
           d=128, n_heads=4,  
           n_layers=4, context=128)
```

Why characters? With GPT-2's 50,257 tokens the embedding table alone would have 6.4M parameters, 8× the rest of our model.

Starting weights: small random numbers, GPT-2's recipe.

TRAINING

Training data = the text shifted by one



F → i
Fi → r
Fir → s · ...

One window of 128 characters holds
128 examples, learned at once
thanks to the causal mask.

A batch = 64 random windows: a
tensor of shape **64 × 128**.

TRAINING

The training loop is the Lesson 02 loop

```
for step in range(1, steps + 1):  
    x, y = get_batch("train")  
    loss = compute_loss(mini, x, y) # 1. FORWARD + 2. LOSS  
    optimizer.zero_grad()          # 5. RESET  
    loss.backward()                 # 3. BACKWARD  
    optimizer.step()                # 4. UPDATE
```

4.17

starting loss = $\ln(65)$: a uniform guess over 65 characters

10.8

GPT-2 would start at $\ln(50,257)$

Loss = cross-entropy of the true next character, averaged over all 64×128 positions, exactly like the classification loss in Lesson 02. Optimizer: AdamW, learning rate 0.001, 3,000 steps.

TRAINING

Watch it learn

Before training · loss 4.18

```
nXbKQhJ
p;H!bAjZE
mwZoLKgpZNhBGwEBP:SAZQnTeQ-c'-
X0uw--
zXqnLf3qZINe:HGY0JvADDzcgPdQWfjRG
uozImEo'e!W$aOSSfWzItI
```

After 300 steps · loss 2.28

```
Histlou! saver,
That dir he sund, ot annd bem fe
tho einte hes thand thaqud
Whake thall thegrl
heere, thancindu
```

After 3,000 steps on a T4

```
[paste a sample from your
Colab run]
```

The same architecture as GPT-2, with about 150× fewer parameters and 40,000× less data. That gap is where today's LLMs come from.

WRAP-UP

From GPT-2 to today's LLMs

	GPT-2 (2019)	today
parameters	124M – 1.5B	up to hundreds of billions
training data	40 GB of text	tens of trillions of tokens
context	1,024 tokens	128k tokens and more
positions	learned wpe	RoPE
normalization	LayerNorm	RMSNorm
MLP	GELU	SwiGLU, mixture of experts
attention	12 full heads	grouped-query attention
after training	nothing	SFT + RLHF → an assistant

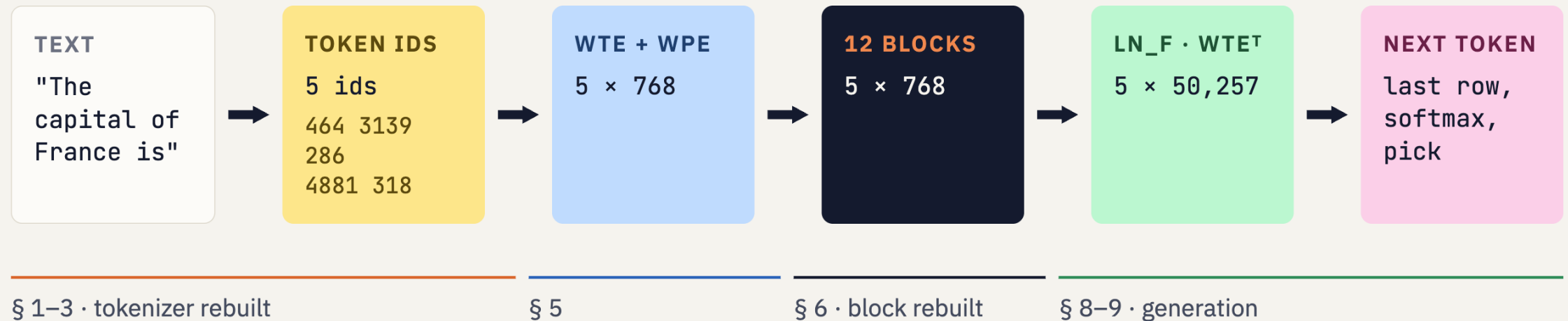
WHAT DID NOT CHANGE

tokens → embeddings →
[attention + MLP, with residuals] ×
N → unembedding

If you understand today's notebook,
you can read the code of a modern
open model.

WRAP-UP

The whole picture, with shapes



Every arrow is a few lines of code in today's notebook, and section 10 trains the whole thing from scratch.

WRAP-UP

Further reading and playing

Transformer Explainer

GPT-2 running in your browser, every step clickable

LLM Visualization

a 3D walk through a small GPT, number by number

The Illustrated GPT-2

Jay Alammar's classic picture book of the architecture

Karpathy: Let's build the GPT Tokenizer

plus [Let's build GPT](#) and [Let's reproduce GPT-2](#)

Tiktokenizer

see how different models cut your text into tokens

Raschka: From GPT-2 to gpt-oss

what has changed in LLM architectures since 2019

THIS WEEK

No new assignment

Finish Assignment 02: your dataset, a fine-tuned model and a public app. Due Oct 15, 18:00.

Play with the notebook: tokenize your own language, find your own attention heads, train on your own text.